

Détection de fraude sur les compteurs intelligents dans le cadre du projet Néovolt

1. Contexte et problématique

Dans le secteur de l'énergie, les fraudes liées aux compteurs électriques représentent une source importante de pertes financières pour les fournisseurs. Elles peuvent prendre plusieurs formes : manipulation physique du compteur, dérivation non autorisée du réseau ou falsification des relevés de consommation.

L'objectif de ce volet consiste à concevoir une solution d'intelligence artificielle capable de détecter automatiquement les comportements suspects à partir des relevés de consommation électrique. Cette détection précoce permet d'optimiser les contrôles terrain, de limiter les pertes économiques et de renforcer la fiabilité du réseau énergétique.

Le cas d'usage retenu est donc la détection de fraude à partir de données historiques de consommation et de caractéristiques des clients.

2. Description des données

Plusieurs sources de données ont été exploitées :

Relevés de consommation

Les relevés contiennent les mesures périodiques de consommation électrique associées à chaque point de livraison (PDL).

Principales variables :

- id_pdl
- date
- consommation_kwh

Informations des compteurs

Les caractéristiques techniques des compteurs permettent d'enrichir les analyses :

- type de compteur
- puissance souscrite
- date de pose
- statut du compteur

Données clients

Les données clients fournissent un contexte sur le profil de consommation :

- surface du logement
- nombre de personnes dans le foyer
- date d'entrée dans le logement

Cas de fraude confirmés

Ces données constituent la vérité terrain (ground truth) utilisée pour l'apprentissage supervisé.

Chaque point de livraison identifié comme frauduleux est utilisé pour construire la variable cible du modèle.

```
DATA_PATH = Path("donnees")

consommation = pd.read_csv(DATA_PATH / "relevés_consommation.csv")
compteurs = pd.read_csv(DATA_PATH / "compteurs.csv")
clients = pd.read_csv(DATA_PATH / "clients.csv")
fraudes = pd.read_csv(DATA_PATH / "cas_fraude_confirmes.csv")

print("Fichiers chargés")
```

3. Préparation et nettoyage des données

La qualité des données constitue une étape essentielle dans la construction du modèle.

Plusieurs opérations ont été réalisées :

Gestion des doublons

Les relevés dupliqués ont été supprimés afin d'éviter l'introduction de biais dans l'apprentissage.

Gestion des valeurs manquantes

Les valeurs manquantes ont été traitées selon leur nature :

- interpolation pour les consommations manquantes ;
- remplacement par zéro pour certaines variables numériques ;
- remplacement par la médiane pour la surface du logement.

Vérification de cohérence

Les consommations négatives ont été éliminées car elles ne correspondent pas à une situation réelle.

```
consommation = consommation.drop_duplicates()

consommation["consommation_kwh"] = pd.to_numeric(consommation["consommation_kwh"], errors="coerce")

consommation = consommation[
    consommation["consommation_kwh"].isna()
    | (consommation["consommation_kwh"] >= 0)
]

consommation["consommation_kwh"] = consommation["consommation_kwh"].interpolate()
consommation = consommation.dropna(subset=["consommation_kwh"])

clients["nb_personnes_oyer"] = clients["nb_personnes_oyer"].fillna(0)
clients["surface_m2"] = clients["surface_m2"].fillna(clients["surface_m2"].median())

compteurs = compteurs[compteurs["statut"] == "actif"]
```

Conversion des types

Les dates ont été converties dans un format exploitable afin de permettre l'extraction de variables temporelles.

```
consommation["date"] = pd.to_datetime(consommation["date"], errors="coerce")
clients["date_entree"] = pd.to_datetime(clients["date_entree"], errors="coerce")
compteurs["date_pose"] = pd.to_datetime(compteurs["date_pose"], errors="coerce")
fraudes["date_detection"] = pd.to_datetime(fraudes["date_detection"], errors="coerce")
```

4. Construction des variables (Feature Engineering)

Le feature engineering constitue une étape déterminante dans la détection de comportements anormaux.

Consommation de la veille

La consommation observée au jour précédent permet de détecter les variations brutales.

Moyenne glissante sur 7 jours

Cette variable représente le comportement habituel du client sur une période récente.

Écart-type sur 7 jours

L'écart-type mesure la stabilité ou l'irrégularité de la consommation.

Variation relative

Le pourcentage de variation entre deux relevés successifs permet d'identifier les ruptures de comportement.

Consommation par mètre carré

Cette variable normalise la consommation en fonction de la taille du logement.

Variables calendaires

Des variables temporelles ont été ajoutées :

- jour de la semaine ;
- mois ;
- indicateur week-end.

Ces informations permettent au modèle de prendre en compte la saisonnalité et les habitudes de consommation.

```
> if "zone" in compteurs.columns:  
|     compteurs = compteurs.drop(columns=["zone"])  
  
df = consommation.merge(compteurs, on="id_pdl", how="left")  
df = df.merge(clients, on="id_client", how="left")  
  
print(df.shape)
```

```

df = df.sort_values(["id_pdl", "date"])

df["conso_j_1"] = df.groupby("id_pdl")["consommation_kwh"].shift(1)

df["moyenne_7j"] = df.groupby("id_pdl")["consommation_kwh"].transform(
    lambda x: x.rolling(7, min_periods=1).mean()
)

df["std_7j"] = df.groupby("id_pdl")["consommation_kwh"].transform(
    lambda x: x.rolling(7, min_periods=1).std()
)

df["variation_pct"] = df.groupby("id_pdl")["consommation_kwh"].pct_change()

df["conso_par_m2"] = df["consommation_kwh"] / df["surface_m2"].replace(0, np.nan)

df["jour_semaine"] = df["date"].dt.dayofweek
df["mois"] = df["date"].dt.month
df["weekend"] = (df["jour_semaine"] >= 5).astype(int)

df.replace([np.inf, -np.inf], np.nan, inplace=True)

```

5. Création du label de fraude

Les identifiants présents dans le fichier des fraudes confirmées ont été utilisés pour construire la variable cible.

La variable fraude prend :

- 1 pour un compteur frauduleux ;
- 0 pour un compteur normal.

Cette approche transforme le problème en classification binaire supervisée.

```

fraudes_ids = set(fraudes["id_pdl"].unique())

df["fraude"] = df["id_pdl"].isin(fraudes_ids).astype(int)

print(df["fraude"].value_counts())

```

6. Détection d'anomalies de référence

Avant la mise en place des modèles de machine learning, une méthode statistique simple a été implémentée.

Une anomalie est détectée lorsque :

Consommation > Moyenne 7 jours + 3 × Écart-type 7 jours

Cette méthode constitue une baseline permettant de comparer les performances des modèles plus avancés.

```
df["baseline_anomalie"] = np.where(
    df["consommation_kwh"] >
    (df["moyenne_7j"] + 3 * df["std_7j"].fillna(0)),
    1,
    0
)

print(df["baseline_anomalie"].sum())
```

7. Gestion du déséquilibre des classes

Les fraudes représentent naturellement une faible proportion des observations.

Pour éviter qu'un modèle prédise systématiquement la classe majoritaire, un équilibrage a été effectué par sous-échantillonnage de la classe non frauduleuse.

Cette stratégie permet d'améliorer la capacité du modèle à identifier les cas rares.

```
df_majority = df[df["fraude"] == 0]
df_minority = df[df["fraude"] == 1]

n_minority = len(df_minority)

df_majority_downsampled = df_majority.sample(n=n_minority, random_state=42)

df_balanced = pd.concat([df_majority_downsampled, df_minority])

df_balanced = df_balanced.sample(frac=1, random_state=42).reset_index(drop=True)

print(df_balanced["fraude"].value_counts())
```

8. Modèles étudiés

Trois algorithmes ont été comparés.

Régression Logistique

Modèle simple, rapide à entraîner et facilement interprétable.

Random Forest

Méthode d'ensemble basée sur plusieurs arbres de décision permettant de capturer des relations complexes.

Gradient Boosting

Algorithme de boosting construisant progressivement un modèle plus performant à partir des erreurs précédentes.

Justification du choix des modèles

Trois modèles ont été sélectionnés afin de comparer différentes approches de classification pour la détection de fraude.

- La **Régression Logistique** a été utilisée comme modèle de référence grâce à sa simplicité, sa rapidité d'entraînement et son interprétabilité.
- Le **Random Forest** a été choisi pour sa capacité à capturer des relations non linéaires et à gérer efficacement des données hétérogènes tout en limitant le surapprentissage.
- Le **Gradient Boosting** a été retenu pour ses excellentes performances sur les problèmes de classification complexes, notamment lorsqu'il s'agit d'identifier des comportements frauduleux rares. La comparaison de ces trois modèles permet de sélectionner la solution offrant le meilleur compromis entre performance, robustesse et capacité de généralisation.

```
models = {  
    "Logistic Regression": LogisticRegression(max_iter=300, class_weight="balanced"),  
    "Random Forest": RandomForestClassifier(n_estimators=200, max_depth=10, class_weight="balanced", random_state=42, n_jobs=-1),  
    "Gradient Boosting": GradientBoostingClassifier()  
}  
  
results = []
```

9. Méthodologie d'évaluation

Les données ont été séparées selon un ratio :

- 80 % entraînement ;
- 20 % test.

Les métriques retenues sont :

Précision : Mesure la proportion de fraudes correctement identifiées parmi toutes les alertes générées.

Recall : Mesure la capacité du modèle à retrouver les fraudes réelles.

F1-Score : Compromis entre précision et rappel.

AUC-ROC : Mesure globale de la capacité de discrimination du modèle.

L'utilisation exclusive de l'accuracy aurait été trompeuse dans un contexte de classes déséquilibrées.

```
for name, model in models.items():
    print(name)

    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    if hasattr(model, "predict_proba"):
        y_proba = model.predict_proba(X_test)[:, 1]
    else:
        y_proba = y_pred
    report = classification_report(y_test, y_pred, output_dict=True)
    auc = roc_auc_score(y_test, y_proba)

    print(confusion_matrix(y_test, y_pred))
    print(classification_report(y_test, y_pred))
    print("AUC:", auc)

    results.append({
        "modele": name,
        "precision": report["1"]["precision"],
        "recall": report["1"]["recall"],
        "f1": report["1"]["f1-score"],
        "auc": auc
    })
```

10. Résultats obtenus

Les trois modèles ont été évalués sur le même jeu de test.

- Regression Logistic

```
[1833 1670]]
```

	precision	recall	f1-score	support
0	0.54	0.61	0.57	3504
1	0.55	0.48	0.51	3503
accuracy			0.54	7007
macro avg	0.54	0.54	0.54	7007
weighted avg	0.54	0.54	0.54	7007

AUC: 0.5574634657573352

- Random Forest

```
Random Forest
[[3132 372]
 [ 269 3234]]
```

	precision	recall	f1-score	support
0	0.92	0.89	0.91	3504
1	0.90	0.92	0.91	3503
accuracy			0.91	7007
macro avg	0.91	0.91	0.91	7007
weighted avg	0.91	0.91	0.91	7007

AUC: 0.9747324781628792

- Gradien Boosting

```

Gradient Boosting
[[3208  296]
 [  37 3466]]
precision    recall  f1-score   support

      0       0.99      0.92      0.95        3504
      1       0.92      0.99      0.95        3503

   accuracy                0.95        7007
  macro avg              0.95      0.95      0.95        7007
weighted avg              0.95      0.95      0.95        7007

AUC: 0.9890568358236971

```

```

results_df = pd.DataFrame(results)
results_df.sort_values("f1", ascending=False)

```

	modele	precision	recall	f1	auc
2	Gradient Boosting	0.921318	0.989438	0.954164	0.989057
1	Random Forest	0.896839	0.923209	0.909833	0.974732
0	Logistic Regression	0.547721	0.476734	0.509768	0.557463

Le modèle retenu est celui présentant le meilleur F1-Score sur la classe fraude.

Dans notre expérimentation, le **Gradient Boosting** a obtenu les meilleures performances globales et a été sélectionné pour le déploiement.

Le modèle final est sauvegardé sous la forme d'un fichier : modele_fraude.pkl

11. Analyse critique

Forces

- Détection automatisée des comportements suspects.
- Utilisation de variables métier pertinentes.

- Comparaison de plusieurs algorithmes.
- Pipeline reproductible.

Limites

- Volume de données limité.
- Équilibrage artificiel des classes.
- Absence de données météorologiques ou contextuelles.
- Risque de dérive du comportement des consommateurs dans le temps.

Risques métier

Un faux positif peut déclencher une enquête inutile.

Un faux négatif peut laisser passer une fraude réelle.

Dans un contexte industriel, ces deux erreurs possèdent un coût opérationnel important.

12. Éléments MLOps

Le projet intègre plusieurs pratiques inspirées du MLOps :

- versionnement du code via Git ;
- sauvegarde du modèle entraîné ;
- séparation des étapes du pipeline ;
- exposition du modèle par API ;
- déploiement cloud.

Une surveillance régulière des performances devra être mise en place afin de détecter une éventuelle dérive des données ou du modèle.

13. Perspectives d'amélioration

Les évolutions futures pourraient inclure :

- intégration de données météorologiques ;
- utilisation de modèles de séries temporelles ;
- mise en œuvre d'un suivi automatisé des performances ;
- détection en temps réel via des flux de données ;

- génération automatique d'explications des anomalies à l'aide d'un assistant basé sur l'IA générative.